

FAEST for Memory-Constrained Devices with Side-Channel Protections

Diego F. Aranha , Johan Degn, Jonathan Eilath, Kent Nielsen and Peter Scholl 

Aarhus University, Aarhus, Denmark

Abstract. We introduce a new compact and constant-time implementation of the FAEST v1.1 signature scheme that allows it to run in resource-constrained Arm Cortex-M4 microcontrollers under 190M cycles for signing or verifying at level 1 security. The main technique for reducing the memory footprint is a new abstraction to reuse or recompute VOLEs on demand, which reduces memory consumption by at least an order of magnitude. Based on the compact implementation, we develop a masked version of FAEST aiming at security against first-order attacks, achieving a performance overhead of 1.26x and a memory overhead of 1.93x. The masked implementation also thwarts horizontal attacks by employing additional shuffling countermeasures. The security of the masked implementation is demonstrated through leakage assessment experiments on the ChipWhisperer platform, both for the main building blocks and the full signature scheme. We conclude the paper by discussing how the side-channel protections can be ported to version 2.0 submitted to NIST.

Keywords: FAEST · VOLE-in-the-Head · Masking · Side-Channel Analysis

1 Introduction

As quantum computing continues to advance, many traditional cryptographic schemes will become vulnerable to efficient cryptanalysis. To address this concern, NIST started a project in 2016 to develop standards for post-quantum cryptography, which culminated with the standardization of hash-based and efficient lattice-based signature schemes. A notable entry to this first process was Picnic 3 [CDG⁺20], built from the MPC-in-the-Head paradigm, and selected as a Round 3 alternate candidate. As an attempt to diversify the hardness assumptions, the initial effort by NIST was followed by the call for additional signature schemes in 2022¹.

FAEST [BBdSG⁺23] is one of the signature schemes which recently reached Round 2 of the follow-up process. It is the only candidate building on the VOLE-in-the-Head (VOLEitH) construction [BBD⁺23], which allows for using the efficient VOLE-based zero-knowledge (ZK) proof system QuickSilver [YSWW21]. FAEST is particularly compelling because it relies solely on the security of well-established cryptographic primitives, specifically AES encryption and the SHA-3 hash function, which are widely regarded as conservative assumptions. FAEST is also built in a modular way, enabling the replacement of its underlying cryptographic primitives, the VOLEitH protocol, or the ZK proof system if any are shown to be insecure.

While certain quantum-safe algorithms can be appealing from a design perspective, their practical deployment may present some obstacles. As standardized post-quantum

E-mail: dfaranha@cs.au.dk (Diego F. Aranha), johantdegn@gmail.com (Johan Degn), eilath@cs.au.dk (Jonathan Eilath), keni@cs.au.dk (Kent Nielsen), peter.scholl@cs.au.dk (Peter Scholl)

¹<https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>

signature schemes will be deployed on hardware with all kinds of limitations, it is important to develop implementations that are efficient on resource-constrained devices. Many post-quantum signature schemes have already been ported to embedded devices, within the `pqm3`², `pqriscv`³ and `pqm4` [KRSS19] frameworks. However, this task can be much more challenging for schemes with a large memory footprint and heavy dynamic allocation. For example, the FAEST reference implementation requires at least 466 KB of memory to compute a signature, and 14 MB in the worst case.

Developing an embedded implementation of a post-quantum signature schemes comes with additional challenges. Inter-operability with the reference implementation is crucial, since changing the parameter set and breaking compatibility would be a major drawback in practice. The threat model is also different, since embedded devices are assumed to be under physical control by an adversary who can mount side-channel attacks to leak secret information, such as based on power consumption [KJJ99] or electromagnetic emanation [QS01]. A common countermeasure to such side-channel attacks is masking [ISW03], where any secret value is secret-shared into $d \geq t + 1$ random shares, and any set of t shares must be statically independent of the secret value, such that an adversary probing up to t shares learns nothing. Considering these challenges, this paper raises and answers the following question affirmatively: *is it possible to make FAEST run efficiently and securely on embedded microcontrollers?*

Contributions. We introduce a new implementation of the FAEST signature scheme that is tailored towards the Arm Cortex-M4 microcontrollers and compatible with reference FAEST signatures. This new implementation comes with a new construction, deemed VOLE Black Box, which enables the on-demand computation of only part of a VOLE, in an oblivious manner to the signing and verification algorithms. The VOLE Black Box is a VOLEitH implementation that is optimized for low memory consumption, at the cost of only a logarithmic increase in computational cost. This new implementation reduces the memory consumption to 52.1 KB of memory for level 1 parameters, including the signature, and runs in constant-time with respect to secret data. To the best of our knowledge, this is the first implementation of a VOLEitH scheme on an embedded device, with a resulting latency in the main variant of 187 million cycles to compute a signature, and 179 million cycles to verify. Due to an order of magnitude lower memory requirement, this implementation additionally increases performance of the reference code by 1.40x for signing and 1.55x for verification on a *consumer-grade machine* due to better utilization of the memory hierarchy, effectively replacing the original reference implementation.

We also note that VOLE-based signature schemes can be vulnerable to side-channel attacks, because the ZK proof requires to manipulate a secret *witness*. We thus consider a probing attack [ISW03] against the FAEST signing algorithm that leaks the secret key to an adversary. The probing attack motivates the need for a masked embedded implementation, thus we introduce new masking techniques for the main building blocks of FAEST. For t -order security, masking the FAEST-specific S-box takes advantage of the inputs being guaranteed to be non-zero to create a masking scheme running in $O(t)$ time and $O(t)$ memory. Masking the QuickSilver ZK proof introduces a memory overhead of $O(t)$, an overhead of $O(t)$ for computing linear operations, and an overhead of $O(t^2)$ for non-linear operations. We illustrate these techniques with a masked implementation of FAEST version 1.1 targeting first-order security, and validate its security using power leakage assessment experiments on the ChipWhisperer platform.

Related work. Few VOLEitH-based signature schemes comparable to FAEST exist, but we expect our contributions to help make others viable on embedded devices. The

²<https://github.com/mupq/pqm3>

³<https://github.com/mupq/pqriscv>

original FAEST authors have proposed a new set of schemes called FAESTER, KuMQuat, and MandaRain[BBM⁺24]. The first scheme utilizes the ability for QuickSilver to prove statements over higher degree polynomials, increasing both speed and communication complexity. The latter two are designed under different hardness assumptions, namely the multivariate quadratic (MQ) problem and the one-way function candidate Rain. The KuMQuat scheme was developed concurrently with another MQ-based signature scheme [Bui24] both significantly reducing the size of the signature while keeping the computational cost competitive.

Our results are easier to compare with other benchmarks at 128-bit security from the pqm4 testing framework. The standardized scheme Dilithium [LDK⁺22] computes a signature in 7.3 million cycles, and a new version of SPHINCS+ called SPHINCS-alpha [YCZ24] requires 382 million cycles. For more directly comparable schemes, an embedded implementation of Picnic 3 [ABE⁺21] computes a signature in 302 million cycles, and MQOM [BFR23] takes 149 million cycles. When compared to other schemes relying on symmetric primitives, our embedded implementation is competitive in terms of speed: only 1.25x slower than MQOM, while being 1.61x faster than Picnic and around 2x faster than SPHINCS-alpha. In terms of memory consumption, signing requires 52 KB for Dilithium, 280 KB for SPHINCS-alpha, and 32 KB for Picnic. Our embedded FAEST implementation uses 42.7 KB, again demonstrating competitive performance.

Now taking masking into account, Picnic [ABE⁺21] and Dilithium [CGTZ23] have masked implementations with a computation overhead up to 5.5x and 53x, respectively. Our masked FAEST implementation has a computation overhead of 1.26x, showing that the masking scheme has a significantly lower overhead. On the memory front, Picnic 3 increased the memory consumption for first-order masking by 280 bytes only, whereas the overhead in our implementation scales linearly with the masking order, showing that with respect to memory the masking approach has significant drawbacks. Recent work by [FRWJ25] suggest a new approach to mask the VOLEitH construction by doing d independent non-masked VOLEitH computations and then combining the results into one. This approach break compatibility with existing implementations making it impractical.

Organization. The rest of the paper is organized as follows: Section 2 gives an overview of the VOLE-in-the-Head framework and the FAEST signature scheme. Section 3 introduces our embedded implementation and the VOLE Black Box technique, together with other optimizations. Section 4 introduces side-channel protections for masking the main components of FAEST, followed by performance and leakage assessment experiments in Section 5. Section 6 discusses the applicability of our results to version 2.0 of FAEST. We conclude the paper on Section 7, including a discussion about how our techniques can be adapted to newer versions of FAEST.

2 Preliminaries

2.1 Notation

For $n \in \mathbb{N}$ we denote $[n]$ to be the set $\{1, \dots, n\}$ and for $a, b \in \mathbb{N} : a \leq b$ we denote $[a, b]$ as the set $\{a, \dots, b\}$ and $[a, b)$ as the set $\{a, \dots, b - 1\}$.

Vectors are column vectors and we denote them using bold letter $\mathbf{x} \in \{0, 1\}^n$ and for matrices we use a capital bold letter, $\mathbf{X} \in \{0, 1\}^{n \times m}$. We use the following notation for indexing:

- $\mathbf{x}[i]$ denotes the i -th element of the vector.
- $\mathbf{X}_i$ denotes the i -th row of a matrix.
- $\mathbf{X}^j$ denotes the j -th column of a matrix.

- $\mathbf{x}||\mathbf{y}$ denotes the concatenation of two vectors.
- $[\mathbf{X} \ \mathbf{Y}]$ denotes the concatenation of two matrices of equal number of rows into a single matrix.

We use the notation $x \leftarrow S$ to sample values uniformly random from a set or distribution, and say that x is sampled from S .

We define the finite fields $\mathbb{F}_{2^8}, \mathbb{F}_{2^{64}}, \mathbb{F}_{2^{128}}, \mathbb{F}_{2^{192}}$, and $\mathbb{F}_{2^{256}}$ as polynomials over $\mathbb{F}_2$ taken modulo an irreducible polynomial. These polynomials are given in the FAEST v1.1 specification [BBdSG⁺23].

2.2 Side Channel Leakage Analysis

We use the probing model [ISW03] to theoretically analyze the side-channel security of the FAEST algorithms. In this model, the adversary is allowed to make t probes within the circuit of computation, where t is known as the order of security. For any c number of executions the adversary has $c \times t$ observations, and can combine them in an arbitrary way to try to extract secret information. The implementation is t -probing secure if the values observed on the t probes are jointly independent of any secret in the computation. We define first-order leakage as the information retrieve by the adversary with $t = 1$ probes.

In addition, we concretely make an assessment of the side-channel leakage of our implementation by conducting a Test Vector Leakage Assessment (TVLA) [GJJR11]. TVLA performs a pass/fail evaluation of whether exploitable leakage is present based on a Welch's t -test. Given two sets of samples of sizes n_1 and n_2 , using their respective sample means $\bar{x}_1$ and $\bar{x}_2$ along with their standard deviations σ_1 and σ_2 , the t -statistic can be computed using the Welch's t -test:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}.$$

The samples are collected by measuring the power consumption of a device during the execution of the implementation at fixed time intervals, according to the sampling rate.

In this work, we use both the fixed-vs-random (FvR) and random-vs-random (RvR) methods. The FvR method tries to detect all possible sources of first-order leakage, by generating two sets of traces, one with a fixed secret key and the other with a random secret key for each signature. The t -test can be used to determine whether the two sets are distinguishable by their means, in which case, the cryptographic implementation has a high probability of containing exploitable leakage dependent on the secret key. The RvR method is used to pinpoint leakage of a specific intermediate value during the computations, by generating two sets of traces, where one group uses one value (e.g 0) and the other group uses another value (e.g 1). If the sets of traces are distinguishable, the implementation has a high probability of containing leakage depending on the specific intermediate value. The threshold for the t -value to determine if the TVLA passes or fails depends on the length of the traces collected. The more sample points the trace contains, the more likely a false positive occurs, hence the threshold is set to have a type I error probability of 0.001 as described by [DZD⁺17]. A type I error happens when the t -value falsely indicates exploitable leakage.

Both the FvR and RvR can be seen as *vertical* side-channel analysis, as the analysis is done on the same observation point across multiple traces. Furthermore, we also consider *horizontal* side-channel analysis, where the analysis is done on multiple observations points across the same trace [CFG⁺10]. Horizontal side-channel analysis can be used to exploit values that are used multiple times in the computation, or secrets that are secret-shared across multiple values. To perform a horizontal side-channel analysis, a

multilayer perceptron (MLP) neural network is used, to predict the secret-shared value based on a single trace.

2.3 FAEST and VOLE-in-the-Head

We briefly review the main components and design of FAEST, which uses the VOLE-in-the-head paradigm [BBD⁺23] for zero-knowledge proofs.

2.3.1 VOLE

An Oblivious Linear Evaluation (OLE) over a field $\mathbb{F}$ is an interactive protocol between a sender and a receiver. The sender provides two inputs $u, v \in \mathbb{F}$ and the receiver gives a single input $\Delta \in \mathbb{F}$. The interactive protocol computes and outputs q to the receiver such that:

$$q = u\Delta - v. \quad (1)$$

The ideal functionality of the OLE protocol states that the receiver only learns Δ, q and nothing about u, v . Likewise, the sender learns nothing about Δ, q .

In Vector OLE (VOLE), the inputs of the sender are a vector of length n and a matrix of size $n \times \lambda$, denoted $\mathbf{u}, \mathbf{V}$, and the output of the receiver is a matrix $\mathbf{Q} \in \mathbb{F}^{n \times \lambda}$, such that

$$\mathbf{Q} = \mathbf{u}\Delta - \mathbf{V},$$

We refer to this as a VOLE relation, or *VOLE correlation* if the inputs $\Delta, \mathbf{u}$ and $\mathbf{V}$ are random. We call Δ the global key, $\mathbf{V}$ for the VOLE tag and $\mathbf{Q}$ for the VOLE key. A VOLE protocol produces triples of the form (u_i, v_i, q_i) such that Equation 1 is satisfied. The tag v_i can be seen as an information-theoretic MAC on the sender's value u_i , under the key (q_i, Δ) known to the receiver. As long as these MACs are under the same global key Δ , they are linearly homomorphic.

2.3.2 VOLE-in-the-Head

VOLE-in-the-Head (VOLEith) [BBD⁺23] is a recent protocol that enables VOLE correlations to be generated non-interactively. The protocol actually implements a relaxation of the VOLE functionality, whereby the sender (or signer) eventually learns the random input Δ of the receiver (or verifier).

All-But-One Vector Commitments. A key component of VOLEith is a form of vector commitment, where the signer can commit to N messages, and then later open $N - 1$ of them to the verifier. When the committed messages are only required to be pseudorandom, this can be done with an opening size of only $O(\lambda \log N)$ bits using a puncturable PRF based on a GGM tree [GGM86] of length-doubling PRGs, together with a hash function.

For $N = 2^d$, the construction works by computing a balanced binary tree of height d , level by level using the PRG, where the master key is the root node of the tree and each node's children are obtained by applying the length-doubling PRG. Each leaf of the tree is then hashed to obtain a pair consisting of a message $\mathbf{sd}_{i \in [N]}$ and commitment $\overline{\mathbf{com}}_{i \in [N]}$. To optimize communication, the leaf commitments are hashed together, to get a single commitment $\mathbf{com}$ for all messages.

From the construction, it is clear that it provides N random messages and a commitment, $\mathbf{com}$, to these messages. After sending $\mathbf{com}$, the signer can later open all messages except the j -th one, by first defining the punctured key consisting of all d siblings on the path from the root to the leaf containing $\mathbf{sd}_j$; these seeds suffice to reconstruct the other $N - 1$ leaves. Then, the opening consists of the punctured key and the commitment $\overline{\mathbf{com}}_j$ for the

missing leaf, which allows the verifier to reconstruct `com` and compare it with the original commitment.

Vector Commitments to VOLE. An $(N - 1)$ -out-of- N vector commitment can be converted into a VOLE over $\mathbb{F}_{2^d}$ between the signer and verifier. Let the unopened index be Δ , which we view as a d -bit integer mapped into $\mathbb{F}_{2^d}$. The messages $\mathbf{sd}_{i \in [N]}$ can be converted into a VOLE of arbitrary length, by using a PRG to expand each seed into a length- ℓ string $\mathbf{r}_i = \text{PRG}(\mathbf{sd}_i) \in \mathbb{F}_2^\ell$, and having the signer define its VOLE outputs as:

$$\mathbf{u} = \sum_{i \in [N]} \mathbf{r}_i, \mathbf{V} = \sum_{i \in [N]} \mathbf{r}_i \cdot i \quad (2)$$

Here, each index i is viewed in $\mathbb{F}_2^d$, and multiplied coordinate-wise with every bit of $\mathbf{r}_i$. The verifier, meanwhile, does not know $\mathbf{sd}_\Delta$, but can compute:

$$\mathbf{Q} = \sum_{i \in [N] \setminus \Delta} \mathbf{r}_i \cdot (\Delta - i) = \sum_{i \in [N]} \mathbf{r}_i \cdot (\Delta - i) = \mathbf{u}\Delta - \mathbf{v}$$

This gives a length- ℓ VOLE correlation over $\mathbb{F}_{2^d}$. The signer and verifier each need to perform $O(N) = O(2^d)$ PRG evaluations, so using this directly to create a large VOLE over $\mathbb{F}_{2^\lambda}$, for security parameter λ , is infeasible. Instead, a better approach described by [Roy22] is used in FAEST where $\tau = \lambda/k$ independent vector commitment instances are constructed, each of size $N = 2^k$, and converts them to τ VOLEs. After sending the commitments, the signer additionally sends $\tau - 1$ correction values, called c_i , to correct the verifier's part of the VOLE correlation so that each VOLE uses the same $\mathbf{u}$ vector. This allows the signer (resp. verifier) to concatenate all its τ $\mathbf{V}$ (resp. $\mathbf{Q}$) matrices to form one length- ℓ VOLE over $\mathbb{F}_{2^\lambda}$.

When putting this all together, the signer first computes a vector commitment hash $\mathbf{com}_i$ for each small-VOLE instance. Then, to save communication, these are all hashed together to define a single commitment $h_{\text{com}} \in \{0, 1\}^{2^\lambda}$ to the τ trees. The signer sends this along with the correction values $c_1, \dots, c_{\tau-1} \in \{0, 1\}^\ell$.

2.3.3 FAEST Overview

FAEST builds a signature following the Schnorr paradigm, via a publicly verifiable zero-knowledge proof of knowledge based on VOLEitH. The key pair $(\mathbf{pk}, \mathbf{sk})$ is defined as $\mathbf{pk} = (x, y)$ and $\mathbf{sk} = k$ such that $E_k(x) = y$, where x is the plaintext, y is the ciphertext, k is the secret and E is a block cipher. In FAEST, the block cipher is instantiated using AES.

A condensed description of the FAEST signing algorithm is shown in Algorithm 1. The signing algorithm works in 4 phases, each phase divided by a Fiat-Shamir challenge. The first phase is the VOLE commitment phase, discussed in the previous section, where the signer computes its VOLE outputs $\mathbf{u} \in \mathbb{F}_2^{\hat{\ell}}, \mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ alongside the commitment, h_{com} . Here, the VOLE outputs are of length $\hat{\ell}$, a parameter slightly longer than the witness length, ℓ , to allow for some additional mask values that are needed. In the second phase, the challenge chall_1 is used to check consistency of the VOLE with the VOLEHash linear universal hash function, which ensures the signer did not cheat (too much) in the c_i correction values. Then, the signer uses the VOLE to authenticate the AES witness $\mathbf{w}$, replacing $\mathbf{u}$. In the third phase, the signer runs the QuickSilver ZK proof system [YSWW21] to prove a set of constraints on the committed witness, and combines these into the result $(\tilde{a}, \tilde{b})$ using the next challenge, chall_2 . In the final phase, chall_3 defines the verifier's VOLE input $\Delta \in \{0, 1\}^\lambda$, which is divided into τ substrings, each used to open one of the all-but-one vector commitments. This allows the verifier to compute its VOLE output and verify the entire zero-knowledge proof.

Algorithm 1: FAEST.Sign(msg, pk = (x, y), sk)	
<pre> 1 $\mu = H_1(pk msg)$; 2 $(r, iv) = H_3(sk \mu \rho)$ // ρ is a random nonce ; 3 $(h_{com}, decom_0, \dots, decom_{\tau-1}, c_1, \dots, c_{\tau-1}, \mathbf{u}, \mathbf{V}) = \text{VOLECommit}(r, iv)$; 4 $chall_1 = H_2^1(\mu, h_{com} c_1 \dots c_{\tau-1} iv)$; 5 $(\tilde{\mathbf{u}}, h_V) = \text{VOLEHash}(chall_1, \mathbf{u}, \mathbf{V})$; 6 $\mathbf{w} = \text{ExtendWitness}(sk, x)$; 7 $\mathbf{d} = \mathbf{w} \oplus \mathbf{u}$; 8 $chall_2 = H_2^2(chall_1 \tilde{\mathbf{u}} h_V \mathbf{d})$; 9 $(\tilde{\mathbf{a}}, \tilde{\mathbf{b}}) = \text{AESProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, pk, chall_2)$; 10 $chall_3 = H_2^3(chall_2 \tilde{\mathbf{a}} \tilde{\mathbf{b}})$; 11 $(pdecom_0, \dots, pdecom_{\tau-1}) = \text{VC.Open}(decom_0, \dots, decom_{\tau-1}, chall_3)$; 12 return $\sigma = ((c_i)_{i \in [1..\tau)}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{\mathbf{a}}, (pdecom_i)_{i \in [0..\tau)}, chall_3, iv)$ </pre>	

Figure 1: Condensed description of the FAEST signing algorithm, taking as input the public key, the secret key and the message to sign, and returning the signature.

Details on AES Expanded Witness. In the second phase of the FAEST signing algorithm, the secret key is expanded to define the witness used in the ZK proof system. Concretely, this witness has the form:

$$\mathbf{w} = (sk || \text{non_lin_word_bits} || \text{shiftrow_bits})$$

Here, $sk \in \{0, 1\}^\lambda$ is the original AES secret key. This is followed by the value of every word that is an output of the non-linear **SubWord** operation run during the key schedule. Then, the witness includes hints about the state during the AES encryption routine, in particular, the state after each **ShiftRows** operation is included, for every round except the last. Since **ShiftRows** is linear and immediately follows the S-box computation, this suffices to give enough information to verify the full AES execution in zero-knowledge.

Details of the ZK Proof. In the third phase of the FAEST signing algorithm, the ZK proof of AES is computed. Using the linearly homomorphic commitment to the expanded witness, the verifier can locally emulate all internal steps of AES except the non-linear component of the S-box, a multiplicative inverse in $\mathbb{F}_{2^8}$. As the expanded witness contains the state after each S-box, the verifier can compute the output of the previous S-box by a backwards computation and the input to the next S-box by a forward computation. To ensure that some input x to the multiplicative inverse is consistent with the output, y , the signer and the verifier use the QuickSilver proof system to verify the constraint $xy = 1$ in $\mathbb{F}_{2^8}$ ⁴. To check these constraints in zero-knowledge, the signer computes a linear combination of the constraints and hides it with some additional VOLE commitments. This allows the verifier to do the same and check the the linear combination of its constraints satisfy the VOLE correlation. This way, the consistency of all S-boxes are verified, convincing the verifier of the signature.

Variants of FAEST. The FAEST signature scheme comes in a series of variants based on the encryption function used, and the parameters of the VOLEitH construction. For each security level $\lambda \in \{128, 192, 256\}$ four variants exists. The variants λf setting $d = 8$ in the VOLEitH construction, and λs setting $d \in \{11, 12\}$ in the VOLEitH construction both uses AES as the encryption function. The variants EM- λf and EM- λs uses the Even-Mansour

⁴To rule out the case that $x = y = 0$, the key generation in FAEST v1.1 uses rejection sampling.

variant, $\text{Rijndael}_{\lambda_x}(k)$ as the encryption function, where the encryption key x is public and the plaintext k is the secret key.

3 Embedded Implementation

This section describes the techniques used to reduce the memory consumption of FAEST in our embedded implementation. This includes a general approach that also applies to other signature schemes based on the VOLEitH paradigm, such as [OTX24, BBGK24]. We also describe additional optimizations which are specific to FAEST on memory-constrained devices. The performance results of the embedded FAEST implementation can be seen in Section 5.

3.1 The VOLE Black Box

The most memory consuming part of FAEST is by far the computation of the VOLE matrix $\mathbf{V}$ (for the signer) or $\mathbf{Q}$ (for the verifier), as detailed in Section 2.3.2. We introduce an abstraction for accessing these matrices in a memory-friendly manner, which we call the VOLE Black Box (VBB), since the signing and verification algorithms remain oblivious to the state of the VBB and how the VOLE is actually computed. This allows for the VBB to be used for all VOLEitH based protocols, enabling the option to run on resource-constrained devices. The VBB consists of a high-level interface for the signing and verification algorithms to access the VOLE, which uses two memory-optimized algorithms: a tree-walk algorithm for vector commitments (VC), and a vector commitments to VOLE computation (VC2VOLE).

3.1.1 Vector Commitment Tree-Walk

Implementing the GGM-based vector commitments according to the FAEST specification [BBD⁺23] gives a memory requirement of more than λN bits. First, observe that the output messages $\text{sd}_0, \dots, \text{sd}_{N-1}$ would require λN memory to be stored. Furthermore, $\overline{\text{com}}_i$ for $i \in [0 \dots N-1]$ must be stored alongside to compute the commitment $\text{com} = \text{H}(\overline{\text{com}}_0, \dots, \overline{\text{com}}_{N-1})$, requiring $2\lambda N$ additional memory. FAEST additionally stores all internal nodes of the GGM tree, to save re-computing the tree during opening, contributing an additional $\lambda(2N-1)$ bits of memory. For the $\lambda = 128$ variants of FAEST v1.1, doing this naively for all τ trees at once requires between 260 kB and 2.9 MB of memory for the fast and small variant respectively.

To reduce this memory cost, we use a tree-walk strategy to incrementally compute the outputs of each vector commitment. By computing a single leaf message, sd_i , at a time, we can improve the memory requirements to $O(\lambda \log N)$, since computing sd_i only requires computing a path from the root of the GGM tree to the i -th leaf. This path is of length $d = \log N$ nodes, thus requires $\lambda \log N$ memory. The leaf hash $\overline{\text{com}}_i$ can be absorbed into com on-the-fly, removing the need to store all the leaf values.

Doing a pre-order tree walk over one GGM tree, the seeds will be output in order, from sd_0 to sd_{N-1} . If the algorithm following the VC consumes these seeds at once, only one seed is required in memory at any given point. If both children of a node are stored when computed, i.e. two nodes in each level, no child will ever be recomputed, hence the tree walk requires the same number of PRG calls as the reference algorithm. This reduces the memory consumption from $O(N)$ to $O(\log N)$ seeds, with no increase in computation (except for the fact that the tree may need to be re-computed later).

Furthermore FAEST utilizes τ GGM trees to compute the VOLE relations. By computing each tree separately, only one tree walk will be in memory at any given point, further decreasing the memory consumption by a factor τ . This approach closely resembles the

streamlining of MPCitH schemes presented in [BBB⁺24], which ensures only one tree is present in memory at once.

At the end of the signing algorithm, the signer needs to compute the openings of the vector commitments. Recall that this is done by revealing one node of the GGM tree on each level except the root, allowing the verifier to recover all-but-one leaves. Since we no longer store the GGM trees in memory, these now need to be recomputed by traversing this path. Note that we do not need to recompute the whole tree, and traversing this path only requires an additional $\log N$ PRG evaluations to compute the required nodes. In total, the tree-walk algorithms for vector commitments slightly increase the number of PRG evaluations by $\tau \log N$, while decreasing the memory consumption from $O(\tau \lambda N)$ to $O(\lambda \log N)$ bits.

3.1.2 Memory-Optimized VC2VOLE

To convert the vector commitments into VOLE (as in Equation (2) for the signer), FAEST uses an optimized divide-and-conquer algorithm from [Roy22] (see also [BBdSG⁺23, Fig. 5.2]). This requires the storage of all expanded seeds, $\mathbf{r}_i$, giving a memory footprint of $N\hat{\ell}$ for each tree. Our approach instead consumes one $\mathbf{sd}_i$ from the tree walk and expands it to $\mathbf{r}_i$ immediately. Then, instead of storing all $\mathbf{r}_i$, we use it to incrementally update the VOLE correlation and forget it. Since the VOLE correlation is a sum of all the expanded seeds, they can all be applied independently of each other, so each column of the signer’s VOLE can be computed using

$$\mathbf{V}^j = \bigoplus_{i=0}^{N-1} \mathbf{r}_i(\lfloor i/2^j \rfloor \bmod 2), \quad \mathbf{u} = \bigoplus_{i=0}^{N-1} \mathbf{r}_i.$$

Here, each expanded seed is added to the j -th column if the j -th bit of the index i is set. Since more than one bit can be set in the index i , each $\mathbf{r}_i$ is applied $d/2$ times on average. This increases the number of XOR operations needed, compared with [Roy22], by a factor $\log N$, but the memory consumption decreases to $\hat{\ell}$. Since the depth, d , of each GGM tree is limited to 12, this performance penalty is minor, because XOR operations are very cheap in practice.

3.1.3 VOLE Black Box Interface

Although the two new algorithms for computing the VOLE relations decrease memory consumption significantly, the VOLE relation itself has a memory footprint of $\lambda\hat{\ell}$. This may be prohibitive for the high security variants of FAEST, e.g. more than 130 kB if $\lambda = 256$. By encapsulating the VOLE relation behind an interface, we can make the VBB only store a subset of the VOLE relation, and recompute different subsets when needed. Using the above techniques, we can now define how the VBB interface works. The interface contains three methods: `init(budget)`, `get_vole(i)`, and `open()`. The `init` function takes a budget parameter for the memory consumption of the VBB. The size of the subset is defined by the budget parameter in the initialization.

In the $\hat{\ell} \times \lambda$ VOLE matrices $\mathbf{V}$ and $\mathbf{Q}$, each row v_i, q_i corresponds to a single OLE where $v_i, q_i \in \mathbb{F}_{2^\lambda}$. If we only want to store a single range of these OLEs, say, from row j to row $j+n-1$, we can modify the conversion to VOLE step, so that instead of computing all $\hat{\ell}$ bits of the expanded seed $\mathbf{r}_i$, we expand only the relevant output starting from the j -th bit. This enables the VBB to store a consecutive subset of the VOLE relations.

We hide this functionality inside the VBB function `get_vole(i)`, which simply returns the i ’th OLE when requested⁵. Thus, the VBB is configured by the budget parameter to

⁵The VBB returns the OLE in column-major order, so when accessing an OLE in the ZK proof phase, the output must be transposed before use.

a fixed memory budget storing a fixed number of OLE's, and only recompute the next part of the VOLE when it is requested. Each time a VOLE is requested that lies outside the current cache, the VBB has to recompute the vector commitment messages using the tree-walk algorithm, to compute the next VOLE batch. Therefore, the memory budget provides a tradeoff between computation and storage costs. This interface ensures that the signing and verification algorithms are independent of the VBB and its configuration, so our approach generalizes to any VOLEitH-based signature scheme. Furthermore, the memory budget of the VBB can be configured independently for the signer and verifier, without any need for coordination.

As the VOLE relation is hidden within the VBB, the `open()` function enables the opening of the vector commitments. The function returns the nodes of the tree that are required to reconstruct the verifier's part of the VOLE relation. These nodes are computed as described in the end of Section 3.1.1.

Since the VBB ensures the underlying algorithms are hidden to the protocol using the VOLE, the algorithms can efficiently be replaced by any further improvement to the VOLEitH construction. This is without any changes to the user of the VOLE, further increasing the modularity of the FAEST signature scheme.

3.2 FAEST-Specific Optimizations

In FAEST, most of the arithmetic is performed on elements of the binary field $\mathbb{F}_{2^\lambda}$. Since $\lambda \in \{128, 192, 256\}$, on constrained devices, this arithmetic can become a performance bottleneck due to the limited word size.

The FAEST reference implementation uses a simple implementation for binary field multiplication that multiplies each bit of one polynomial with the other and reduces the result each time. Consequently, binary field multiplication constituted a significant portion of the total runtime on the embedded device. Thus, we implemented an optimized algorithm for binary field multiplication, combined with fast polynomial-specific implementations for modular reduction.

Furthermore when using the VBB with a reduced storage size, part of the VOLE output has to be recomputed each time a requested OLE is not present in the cache. To avoid recomputing the same OLE multiple times, we modified the ZK proof implementation resulting in an (almost) linear access pattern, reducing the amount of recomputations to a minimum as each OLE is only computed once.

3.2.1 Binary Field Arithmetic

We replaced the existing implementation of binary field arithmetic with the left-to-right López-Dahab comb method as proposed in [LD00]. It precomputes a table of size 2^w with the products of small polynomials and the first input polynomial. Splitting the second input polynomial in chunks of size w , each chunk is interpreted as a small polynomial and the resulting polynomial retrieved by a lookup in the table. The retrieved values are shifted according to the corresponding chunks offset in the second polynomial and added together. The result is a polynomial of degree at most double any of the input polynomials.

To reduce the resulting polynomial we implemented highly optimized polynomial-specific modular reductions for each of the polynomials, exploiting their sparsity to perform just a few shifts and additions. This classical technique allows the reduction to be performed efficiently by rewriting the higher degree coefficients into lower degree coefficients multiplied by the low-degree terms of the irreducible polynomial. This approach is further simplified by the fact that the degrees of all irreducible polynomials are powers of two, making it possible to fit both the unreduced and reduced polynomial into a whole number of machine words.

3.2.2 Linear VOLE Access

Due to the ability of the VOLE Black Box to store only a subset of the VOLE, and the fact that each recomputation comes with a cost, the *access pattern* with which the VOLE is used now has a crucial impact on performance. By access pattern, we refer to the sequence of indices requested from the VOLE. Clearly, random access to the VOLE will become very costly since the VBB only stores a consecutive subsection of the VOLE, hence can introduce a large number of recomputations. We have therefore modified the AESProve algorithm to have an almost linear access pattern of the VOLE to minimize the number of recomputations.

As described in Section 2.3.3, the VOLE is used to create linear homomorphic commitments to the extended witness, which is based on the internal state of the AES key schedule and encryption algorithms. Thus, the order of how these commitments are made dictates how the VOLE is accessed. The reference implementation — following the specification — commits to all the forward S-box computations first, and then the backward computations afterwards. However, since the forward and backward computation originates from the same state, the VOLE elements needed to compute the commitment are based on the same OLEs, hence each OLE is accessed multiple times. Furthermore, the OLEs needed to commit to the round keys, are used in both the key expansion and during the encryption. This non-linear access pattern is shown in Fig. 2a.

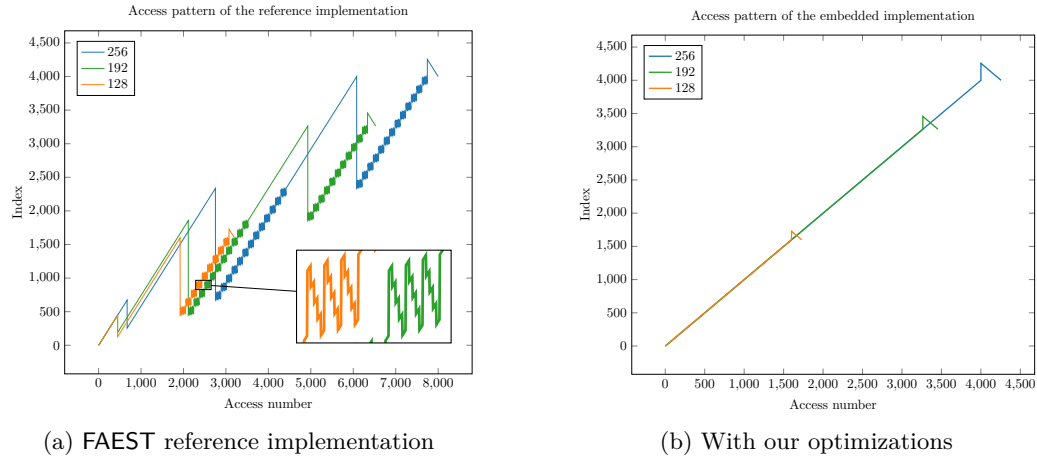


Figure 2: Comparing access patterns to VOLE in the ZK proof.

We solve this problem with two ideas. First, we interleave the forward and backward computations such that they are computed simultaneously, with each state is only committed to once. This is done by committing to state i , using the respective OLEs needed, then from state i the backward computation computes the output of the previous S-box, and simultaneously the forward computation computes the input to the next S-box. State i will now never be accessed again, and likewise for the corresponding OLEs. By computing the input and output of the S-boxes state-by-state, we eliminate the need to store all the inputs and outputs at the same time. As soon as one state has been computed, we can also compute the relevant ZK constraints and absorb them into the linear combination to be checked. This not only creates a linear access pattern of the VOLE, but also decreases the memory usage of the ZK proof by a factor R , where R is the number of rounds in the AES encryption.

Secondly, for the key expansion phase, we allocate dedicated memory to a cache for the OLEs used to commit to the round keys. This avoids recomputations each time a round key is used during the encryption. This cache will contain ℓ_{KE} OLEs, which is the number

of OLEs needed to commit to the round keys that are outputs from the S-boxes. Since a round-key must be added to each state, we remove the need to recompute these OLEs by storing them in the dedicated cache. By caching these OLEs, we further reduce the memory consumption of the ZK proof by not storing commitments to all round keys, but computing each commitment as needed based on the cache. This has a small performance penalty, but reduces the memory consumption by $O(\lambda R)$.

Our final access pattern to the VBB is shown in Fig. 2b. Note that it is completely linear, save for the final λ OLEs used to mask the ZK constraint, which are accessed in reverse order. We can easily handle these λ OLEs without any recomputation, by treating these indices as a special case in the VBB implementation.

4 Side Channel Protections

The embedded implementation is trivially prone to probing attacks as values dependent on the secret key are manipulated throughout the computation. This motivates the need for a masked implementation, of which we will explain techniques used to prevent side-channel leakage. The masked implementation uses Boolean masking to compute the signature on secret-shared values, protecting the implementation against side-channel leakage while ensuring compatibility with the reference version. Our embedded and masked implementations were integrated into the `pqm4` framework and made available as open-source software ⁶.

4.1 Scope of Masked Implementation

We have implemented a masked version of the FAEST signing algorithm under a limited threat model, in which we allow a t -probing adversary to learn t bits of ephemeral values that cannot be reused across multiple signing operations. This is the same approach taken in [ABE⁺21], as a way to reduce the performance penalty of masking hash function calls. Each step of Algorithm 1 will be analyzed with respect to a probing adversary.

The initial hash (H_3) of the secret key to derive the seed for the GGM tree is masked using the masked SHA3 implementation from [ABE⁺21]. Assuming that the secret is stored in a Boolean secret sharing, the masked hash function can be directly applied.

The VOLEitH construction (described in Section 2.3.2) is randomized per signature, since the root key is derived from the signature randomness. This implies that an attacker who can probe t bits from the seed in a single computation decreases the security of the signature by t -bits. When the attacker probes the next computation, it will have a new random seed independent of the previous one, hence independent of the t bits learned from the previous computations. The same argument applies to all intermediate nodes in the GGM trees, alongside the conversion to the VOLE relation, thus the VOLE Black Box is left unmasked.

The computation of chall_i using hash functions H_2^i is not masked, as a probing adversary learns no secret by probing these challenges. The input and output of any challenge are public values, which can be derived using the verification algorithm, thus computing the hash in the clear does not pose any risks. The `VOLEHash` in step 5 is left unmasked, since the VOLE relation is random and independent for each computation of the signature.

In step 6, the witness $\mathbf{w}$ used in the ZK proof is derived using the public key and secret key. This derivation must be masked, since the witness is deterministically computed from the secret key, hence constant across all computations. A probing attacker will be able to learn the secret key, t bits at a time, by probing the witness. The same applies to the ZK proof itself in step 9 of the signing algorithm, where the proof is derived from the

⁶<https://github.com/johandegn/pqm4-faest/>

Algorithm 2: S-box-Masked(x_0, x_1) : $x = x_0 \oplus x_1$
<pre> 1 $r \leftarrow \{2^8\} \setminus \{0\}$; 2 $t_0 = r \cdot x_0$; 3 $t_1 = r \cdot x_1$; 4 $t = t_0 \oplus t_1$; 5 $y = t^{-1}$; 6 $r' \leftarrow \{2^8\}$; 7 $y_0 = \text{Affine}(r \cdot (y \oplus r'))$; 8 $y_1 = \text{Affine}(r \cdot r')$; 9 return (y_0, y_1) </pre>

Figure 3: Our FAEST specific S-box gadget, taking as input a secret shared value x and outputting the inverse, y , of x as a secret shared value. The construction of the gadget is insecure against the input values being zero, which by design of FAEST is not possible.

witness and the VOLE relation. In total two parts of the signature needs to be masked: the witness derivation and the ZK proof.

4.2 Masking the Witness Derivation

In FAEST the extended witness $\mathbf{w}$ is computed by extracting states from the AES key expansion and encryption. Implementing masked AES has been the subject of many research papers, with the main difficulty is implementing the masked S-box, since it is the only non-linear operation in the algorithm. The AES S-box is defined as the multiplicative inverse in the field $\mathbb{F}_{2^8}$, with the non-invertible zero element being mapped to itself, followed by an affine transformation over $\mathbb{F}_2$.

We propose a new masked S-box gadget specifically designed for FAEST. The FAEST key generation ensures that the input to all S-boxes in the proof are non-zero, hence the S-box gadget does not need to handle this special case. This allows the gadget to transform the Boolean masking into a multiplicative masking, which allows for simple implementation of the multiplicative inverse.

The algorithm for the S-box gadget is shown in Figure 3. Lines 1-4 convert the Boolean masking into multiplicative masking, by multiplying the shares over $\mathbb{F}_{2^8}$ with a random non-zero value r . The resulting values $t = r(x_0 \oplus x_1) = r \cdot x$ is still first-order masked, since probing of any of $\{t, r\}$ does not reveal enough information to derive x . Clearly r must not be zero, otherwise it would destroy the secret value, but it is less subtle that x must also be nonzero. If this was the case t_0, t_1 would result in shares of the same value and t would become zero, hence revealing the fact the x is zero. As this can never happen in FAEST by design, the masking conversion is first-order secure. Lines 6-8 then perform the conversion from multiplicative masking back to Boolean masking, by removing the random value r . It also applies the affine transformation to the new secret sharing of the inverse. Implementing the multiplicative inverse in line 5 can be done in a non-masked fashion, e.g using a lookup table, since the input is masked using a multiplicative mask. By applying the inverse, the output will be $t^{-1} = r^{-1} \cdot x^{-1}$, which allows for simple removal of the random value r in the conversion back to Boolean masking.

Using the FAEST specific S-box gadget, we have implemented the masked derivation of the witness $\mathbf{w}$ by computing the AES encryption on secret shared values, with each invocation of the S-box replaced by the S-box gadget. This enables the masked implementation to take the secret key in secret-shared form, and produce the witness in secret-shared output. The masked witness part of the signature $\mathbf{d}$ can be computed directly from the

masked witness, by

$$\mathbf{d} = (\mathbf{w}_0 \oplus \mathbf{u}) \bigoplus_{i=1}^t \mathbf{w}_i,$$

where $\mathbf{u}$ is the VOLE secret, and $\mathbf{w}_i$ is the i -th share of the witness. The order of operations highlighted in the equation is important, to ensure the witness is not accidentally reconstructed.

4.3 Masking the ZK-Proof

The ZK proof of FAEST proves that the signer can compute the public key using an AES encryption of the secret key. This works similarly to the witness derivation. However, as the ZK proof is unable to handle the non-linear operation of the S-box, the proof produces constraints that these S-boxes are computed correctly. Thus, the ZK proof of AES only involves the linear operations, and can therefore be computed independently on each share of the input. The only non-linear operation is deriving the constraints of the S-boxes, which are computed as:

$$\begin{aligned} A_{0,j} &= \mathbf{V}_s[j] \cdot \mathbf{V}_{\bar{s}}[j], \\ A_{1,j} &= (s[j] + \mathbf{V}_s[j]) (\bar{s}[j] + \mathbf{V}_{\bar{s}}[j]) - 1 - A_{0,j}. \end{aligned}$$

This computation includes the product between secret-shared values. Even though the VOLE is randomized per signature, and thus not masked, the secrets ($s[j]$ and $\bar{s}[j]$) will be equal for all signatures, thus the product must be computed in masked form. This multiplication is implemented using the secure multiplication protocol in $\mathbb{F}_{2^n}$ proposed in [RP10]. This secure multiplication uses $(t+1)^2$ field multiplications alongside $t(t+1)/2$ random field elements, which are a significant overhead. However, due to the fixed number of constraints and the ease of implementation, this is a good trade-off.

The constraints of the ZK proof are hashed using the ZKHash function, which is a linear hash function. This enables the hash to be straightforwardly computed on secret shares and produce the results in secret-shared form. Strictly speaking, the ZKHash function does not need to produce a secret-shared output, since the output is public and used by the verifier, hence the outputs are reconstructed and placed in the signature.

4.4 Overhead of Masking

Masking the signing algorithm has associated computation and memory costs. We first look at the computation overhead of the masking, and later discuss the memory overhead.

The witness derivation and ZK proof consist mostly of linear operations, making their masking overhead directly proportional to the order of security. The S-box gadget is also linear with respect to the masking order. This is because the cost of converting to and from multiplicative masking is proportional to the number of secret shares, while the inversion itself is computed with fixed cost. The multiplication of the constraints in the ZK proof is quadratic with respect to the number of shares, giving the masking an asymptotic $O(t^2)$ overhead. However, as only a small part of the computation is spent on the proof constraints, the quadratic overhead does not have a significant impact on the overall performance.

As described in Section 4.1, the VOLEitH construction itself is not part of the threat model, hence it is not masked. Protecting the VOLEitH construction would require a masked PRG, which in turn would require a masked implementation of AES-CTR. The fastest known masked AES implementation on microprocessor at the time of writing is the Fixsliced implementation by Adomnican and Peyrin [AP21], giving an overhead of 3.8x for first-order masking. This would directly result in a 3.8x overhead for the VOLEitH construction. As the VOLEitH construction takes 75% of the computation, the overall

performance penalty for masking VOLEitH would be 2.85x. For higher orders of masking no known implementations are available, however [CGP⁺12] showed that the asymptotic complexity of the S-box computation will be $O(t^2)$ for higher orders of masking, hence the AES implementation will have a similar asymptotic overhead. This will have a dramatic performance penalty, since VOLEitH is the majority of the computation.

The storage overhead for masking is the memory needed to store the secret-shared values. As the masked implementation uses a Boolean masking, the memory consumption is directly proportional to the order of masking. This pairs well with the VOLE Black Box as the memory consumption can be controlled to fit the available memory on the target device.

4.5 Horizontal Side-Channel Analysis

It was shown in an attack on NewHope[ACLZ20], that computation using a bit mask which is either all 0's or all 1's generate sufficient side-channel leakage to allow key recovery. Since the bit mask is dependent on the value of individual bits of the secret, each bit can be extracted using a machine learning model, resulting in extraction of the secret key from a single trace. Such a horizontal analysis can use the model to learn values from different sample point of a single trace in the signature computation. Concurrent work by [JD25] shows that a similar attack is possible on two parts of the reference implementation of FAEST with boolean masking enabled. They show using the same approach as the attack on NewHope, that each bit of both secret shares can be extracted due to leakage depending on an individual bit. This extraction effectively circumvents Boolean masking. We present how the attack works in the reference implementation of FAEST and how the masked implementation uses the countermeasures suggested by [JD25] to remove this type of leakage.

The first affected part in the reference FAEST code, is the conversion from $\mathbb{F}_2^8$ to $\mathbb{F}_{2^\lambda}$ in the `byte_combine_bits`. It uses a similar bit mask to increase the efficiency of the conversion. The conversion is done through a generator $\alpha \in \mathbb{F}_{2^\lambda}$ multiplied by each bit of input x :

$$y = \sum_{i=0}^7 \alpha^i \cdot x[i]$$

The bit mask is created by extending the value of $x[i]$, such that the multiplication can be done efficiently. A horizontal analysis is then able to learn the value of $x[i]$ for all i , resulting in the extraction of x . The analysis can be extended to a Boolean masked implementation by extraction of each share of x .

The masked FAEST implementation removes the bit dependent leakage by removing the need for a bit mask. This is done by computing conversion using conditional moves instead. Because $x[i]$ is a single bit, a conditional multiplication will still create leakage correlated to the value of $x[i]$, since the result is either α^i or 0. By computing both cases for $x[i]$ and selecting the correct one, the arithmetic operations performed become independent of $x[i]$ regarding both value and timing. To select the correct output the arithmetic conditional move by Scott [SS25] is used. This conditional move can be done without conditional execution, using random values to mask both the computation and the selection. The idea is to select between values x and y based on bit b by evaluating $x = ((1 - b) + r)x + (b + r)y - r(x + y)$ with a random mask r . Horizontal analysis cannot be performed on b effectively, as the computation is independent of b except the arithmetic conditional move. If the conditional move is computed within registers and never stored, the horizontal attack will be unsuccessful as the model is unable to distinguish the change in value as suggested by [JD25] and experimentally shown in Section 5.3. Hence by selecting between 0 and α^i , by setting $b = x[i]$ the leakage of the conversion becomes independent of the input x .

The second affected part is the on-demand transpose of each OLE tag during the ZK proof. The reference implementation performs this computation bitwise, by extracting the bit on position (i, j) and writing it to position (j, i) , one by one. An adversary using a horizontal side-channel attack, can train a model to learn the value of the bit being transposed. Since the verifier of a signature can compute q, Δ in Equation 1, learning only a single bit of the transposed OLE tag, $\mathbf{v}$, can reveal the value that it is hiding, $\mathbf{u}$. Since the OLE is over the field $\mathbb{F}_{2^\lambda}$, the subtraction operation is a bitwise XOR, hence from a single extracted bit it is easy to compute the hidden value. By using the same model on multiple bits will increase the success rate of the extraction. This extraction has to be performed on both shares of the OLE since the implementation use a Boolean masking. Such an attack is captured by the probing model, as in this model the adversary cannot probe all the shares.

The masked FAEST implementation transposes multiple bits of each OLE at a time to remove the bit dependent leakage that can be exploited. As each bit of the OLE is not placed consecutively in memory, the transpose works by loading the relevant values into a register and then performing a series of rotation operations to place the bits consecutively. By doing this enough times to write the transpose OLE as bytes instead of bits, the individual bits are never handled separately. An adversary using a horizontal analysis can attempt to extract information from this transpose in multiple places, however operations are masked by bits of different OLEs, thus the values manipulated does not only depend on a multiple bits of the OLE of interest, but additional bits from other OLEs than the targeted.

Concretely, the loading of one bit from a OLE in memory an additional 7 different bits from 7 different OLEs will be returned. Combining multiple bytes into a machine word, the individual bits of the OLE are both masked by other bits from the same OLE and 7 different OLEs. As the operations are rotations of a machine word, the Hamming weight of the operation will be constant regardless of the order of the bit, thus the leakage will be dependent on all bits simultaneously.

Shuffling. The countermeasures against horizontal analysis will still produce unavoidable leakage since values must be manipulated. For the conversion from $\mathbb{F}_2^8$ to $\mathbb{F}_{2^\lambda}$ the best scenario is leakage correlated to the value of the input. Independent of the implementation of the conversion the attack by the horizontal analysis can simply be performed on the input before the conversion begins. In practice this is leakage correlated to the value of the byte, e.g the hamming weight. This leakage will drastically reduce the complexity of key recovery, as leaking the Hamming weight greatly reduces the number of possible values the secret can take. Hence, to ensure the complexity of the key recovery attack is as large as brute force, shuffling is applied during the computation of the signature. The masked implementation uses a constant-time implementation of the fast and unbiased Fisher-Yates [FY38] shuffling algorithm.

Looking at the attack on `byte_combine_bits`, the function is used during the ZK proof to both lift the secret key and the round keys to the field $\mathbb{F}_{2^\lambda}$. Hence, only 4 bytes are lifted during key expansion and 16 bytes are lifted during encryption. Shuffling the order of these bytes will produce less than 2^5 and 2^{45} possible combinations respectively, which are much faster to brute-force than exhaustive key search. To increase the complexity a series of dummy calls to `byte_combine_bits` are done alongside the actual calls lifting the keys. By shuffling together the dummy and actual calls alongside the order of the bytes, the complexity increases dramatically. When the secrets are lifted during key expansion, a total of 9 dummy call are added, increasing the possible combinations to $(4 + 9)! \geq 2^{32}$. When the secrets are lifted during encryption, the 16 bytes of the state are first shuffled and then each call is shuffled with 5 dummy calls, increasing the possible combinations to $((5 + 1) \cdot (5 + 1))^{16} \cdot 16! \geq 2^{126}$. The first term of the product is the number of possible combinations across each byte of the encryption state. For byte i in the first share and

byte j in the second share, the number of valid combinations is $(6 \cdot 6)$ as both contain 6 calls. Now for each selection of the 6 calls in byte i , additional $(6 \cdot 6)$ combinations are possible for byte $i + 1$, resulting in the total over all 16 bytes to be $(6 \cdot 6)^{16}$. At last, the order of the bytes is shuffled, leading to the $16!$ combinations even if the secret shares are combined.

Because the 4-byte case can reveal at most 32 bits of the secret key with attack complexity greater than 2^{32} , exhaustive key search is just as efficient as attacking the shuffled shares. Likewise, the 16-byte case can reveal the full round key with attack complexity greater than 2^{126} , which is almost as expensive as exhaustive key search.

A similar shuffling idea has been applied to the transposing of the OLEs. Each OLE consists of 16 bytes, hence transposing each byte can be shuffled together to increase the complexity of the attack. Furthermore, by transposing 16 OLEs for each invocation, the order of the OLEs can be shuffled to further increase the combinatorial complexity. The only cost of transposing 16 OLEs instead of one for each invocation is an additional memory consumption of 240 bytes and the cost to compute the shuffling. The masked implementation uses a binary mask to shuffle the bytes and a permutation to shuffle the OLEs, giving a total of $2^4 \cdot 16! \geq 2^{48}$ combinations to give valid reconstructions of the OLEs. Since each OLE masks a single bit of the secret, 16 OLEs can leak at most 16 bits of information, thus the combinatorial complexity of 48 bits of entropy is at first sight much greater than needed. However, it is much easier to check whether a share reconstruction is correct, since each bit of the OLE tag $\mathbf{v}$ must be consistent with a fixed $\mathbf{u}$ and the known Δ , and thus not all 2^{48} possible reconstructions are needed to be fully checked to reconstruct the OLEs correctly. As the actual number of valid reconstructions depends on the values of the secret shares alongside the values of the OLE, the 32 bits of additional combinatorial complexity are added as a safety margin to ensure the attack is infeasible. The safety margin can be increased much further by transposing additional OLE in each invocation. Increasing the number of OLEs to 40 would increase the safety margin to 131 bits with only 384 bytes of memory penalty.

Fault Injection. In the work by [JD25] two fault injection attacks on the reference implementation is presented. These are not addressed in this work, as they are out of the scope for the protected implementation. The authors do present countermeasures to the fault injection attacks, by letting the signer verify the signature before communicating the signature to the verifier. This countermeasure solves the issue presented and can be done without masking, as the verification only includes public information. The computational overhead of this countermeasure is easily predictable by combining both the signing and verification costs based on our benchmarks in the next section. The memory overhead will not change, since all data from signing can be forgotten when the signature is verified.

5 Experiments

In this section, we present the experimental results of our FAEST implementation⁷. We first report the benchmarks for the embedded implementation, and later present the TVLA results for the masked version.

The masked implementation is designed for the Arm Cortex-M4 processor and uses the on-board True Random Number Generator (TRNG) to generate the randomness needed for masking. Handling each share of any secret shared value securely should in theory prevent any leakage during computation. However, transitional leakages occur when secret values are accidentally combined or reconstructed by the compiler [GD23]. The main transitional leakages are due to memory pipeline registers, overwriting shares in registers, or leftover

⁷<https://github.com/johandegn/pqm4-faest/>

values on the stack from previous computations. Some functions, such as the S-box gadget and the ZKHash, must manipulate the shares and are therefore implemented in assembly to ensure no transitional leakage occur. Furthermore, any loading or storing of secret values is followed by a dummy load or store to ensure that the memory pipeline contains non-secret information, and registers are cleared after holding secret values. Likewise, when a function with access to a secret share is followed by the same function applied to another share, the stack is cleared to remove interference between temporary variables. All memory allocations have been changed to stack allocations, allowing the `pqm4` framework to accurately measure memory usage.

It is important to note that the implementation both for desktop and embedded does not contain any optimizations that are architecture specific. The benchmarks serve to highlight the performance changes due to the alteration to reduce the memory consumption, and give a indication on the performance on embedded devices along side the masking overhead.

5.1 Benchmarks

5.1.1 Embedded Compared to Reference

We first compare the new embedded implementation to the reference implementation on a consumer-grade laptop. This highlight the decrease in memory consumption and the algorithmic changes from the VOLE Black Box. This platform is chosen for comparison since not all variants for the reference implementation fit on the embedded platform

The implementations were built using the XKCP [Tea21] implementation of SHA3, and AES from the OpenSSL library, which utilizes hardware instructions for better performance. They were compiled and tested on an x86-64 AMD Ryzen 7 4750U processor. This processor has 64 KB of L1 cache and 512 KB of L2 cache per core and a shared 8 MB L3 cache. They were compiled with GCC 14.2.1 using the `-O3` optimization flag. Each variant of the two implementations was executed one at a time and repeated 100 times to minimize noise. The results are shown in Table 1. The memory consumption figures include the size of the signature.

Across all variants and security levels, the memory requirements of the embedded implementation are significantly lower than the reference implementation, ranging from one to two orders of magnitude. Additionally, the embedded implementation is faster than the reference implementation for all variants and security levels. We conjecture the speedup is due to the much lower memory footprint, allowing computations to fit within a low-level cache, which greatly decreases the time spent on memory access. The optimization of interleaving and linearization of the access pattern discussed in Section 3.2.2 further contribute to better cache utilization. The optimized binary field multiplication also contributes to the overall performance, especially for the high-security variants, but most of the speedup is most likely due to lower memory footprints. Due to the overall better performance of the embedded implementation of the signature scheme, our new implementation effectively replaces the reference implementation of FAEST.

5.1.2 Performance and Memory Usage on Embedded

We now present the performance and memory usage of the embedded implementation running on an embedded device. The benchmark has been performed using the `pqm4` framework on the Nucleo-L4R5ZI board, which has 2 MB flash and 640 KB RAM and is based on the 32-bit Arm Cortex-M4 architecture. All benchmarks have been performed with full cache size for the VBB.

The memory consumption and average latency in CPU cycles for the embedded implementation on the Nucleo-L4R5ZI board are shown in Table 2. Note that the memory

Table 1: Performance and memory benchmarks for 100 runs on a consumer grade laptop. The memory consumption for both signing and verification include the signature.

	Scheme	Memory Consumption			CPU Cycles		
		KeyGen	Sign	Verify	KeyGen	Sign	Verify
This work	128f	7.74 KB	52.1 KB	51.9 KB	790 K	60 M	59 M
	128s	7.74 KB	50.9 KB	50.8 KB	692 K	321 M	322 M
	192f	7.74 KB	127.7 KB	124.5 KB	845 K	113 M	98 M
	192s	7.74 KB	123.7 KB	120.8 KB	912 K	664 M	647 M
	256f	7.74 KB	208.8 KB	204.3 KB	964 K	143 M	133 M
	256s	7.74 KB	202.5 KB	198.4 KB	984 K	834 M	811 M
Ref.	128f	7.74 KB	514.0 KB	195.4 KB	835 K	66 M	60 M
	128s	7.74 KB	4.17 MB	2.30 MB	819 K	337 M	347 M
	192f	7.74 KB	1.09 MB	391.3 KB	1049 K	498 M	461 M
	192s	7.74 KB	11.5 MB	4.35 MB	1062 K	1026 M	1015 M
	256f	7.74 KB	1.78 MB	528.0 KB	1325 K	732 M	684 M
	256s	7.74 KB	14.0 MB	5.47 MB	1266 K	1401 M	1362 M
This work	EM-128f	7.74 KB	38.8 KB	39.3 KB	740 K	55 M	53 M
	EM-128s	7.74 KB	37.7 KB	38.1 KB	746 K	314 M	309 M
	EM-192f	7.74 KB	88.7 KB	91.9 KB	698 K	87 M	89 M
	EM-192s	7.74 KB	85.6 KB	88.8 KB	777 K	615 M	597 M
	EM-256f	7.74 KB	171.9 KB	177.0 KB	928 K	138 M	132 M
	EM-256s	7.74 KB	166.2 KB	171.3 KB	784 K	804 M	784 M
Ref.	EM-128f	7.74 KB	474.6 KB	169.2 KB	749 K	59 M	54 M
	EM-128s	7.74 KB	3.76 MB	1.98 MB	763 K	329 M	323 M
	EM-192f	7.74 KB	1.01 MB	303.9 KB	923 K	327 M	355 M
	EM-192s	7.74 KB	10.5 MB	3.37 MB	937 K	823 M	859 M
	EM-256f	7.74 KB	1.74 MB	486.4 KB	1135 K	602 M	632 M
	EM-256s	7.74 KB	13.8 MB	5.04 MB	1233 K	1245 M	1291 M

usage is excluding the signature size, differently from Table 1, following the values reported by `pqm4`. The reference implementation has not been benchmarked on the embedded device, as its memory consumption is too large for most variants. All variants of the embedded implementation are running in constant-time with respect to the secret key. The non-masked variants have a variable cycle count, due to incrementing the iv and opening the GGM trees. Incrementing the iv depends on the number of machine words needs to be change and and opening the GGM trees depends on the bits of Δ . There is no security issue regarding the timing of the computation, since both values are public. The masked implementation uses constant-time implementations for these operations, which have a minor performance impact.

There is a direct correlation between the security level and both the memory consumption and the latency, so increasing the security level also increases the computational cost. The least expensive variant is the EM-128f, running in 31.3 KB of memory and 158 M cycles for signing and 31.1 KB and 154 M cycles for verification. This highlights the small memory footprint in relation to the overall memory available on the benchmark device, showing that the implementation can be run on devices with more restricted resources. Reporting the memory consumption excluding the signature gives a practical number, since the signature can be streamed out as soon as it is computed. The computation cannot enter a fail state invalidating part of the signature, hence if the device fails in some way, only a subset of a valid signature can be revealed, which will not degrade security. The memory usage is almost equal between the fast and small variants, which argues for the fast variant to be used on resource-constrained devices due to the significantly better performance. If the device have access to a hardware implementation of AES, the unmasked VOLEitH computation will be significantly faster for all variants, reducing the

Table 2: Memory usage (w/o signature) and average CPU cycles based on 100 executions for fast variants and 25 executions for small variants. The “Random bytes” column measures the amount of randomness needed for computing a signature. The non-masked variant uses λ bits to generate ρ .

Scheme	Stack Consumption			CPU Cycles			Random bytes
	KeyGen	Sign	Verify	KeyGen	Sign	Verify	
128f-masked	–	82.5 KB	–	–	235 M	–	250 KB
128f	808 B	42.7 KB	41.4 KB	82.5 K	187 M	179 M	16 B
128s	808 B	42.7 KB	41.7 KB	82.5 K	1392 M	1353 M	16 B
192f	808 B	108.3 KB	105.6 KB	385 K	538 M	516 M	24 B
192s	808 B	108.3 KB	105.6 KB	446 K	4060 M	3931 M	24 B
256f	808 B	178.0 KB	174.8 KB	614 K	800 M	762 M	32 B
256s	808 B	178.0 KB	174.5 KB	606 K	5756 M	5576 M	32 B
EM-128f	808 B	31.3 KB	31.1 KB	75.3 K	158 M	154 M	16 B
EM-128s	808 B	31.3 KB	31.2 KB	80 K	1288 M	1256 M	16 B
EM-192f	904 B	73.1 KB	75.7 KB	225 K	363 M	354 M	24 B
EM-192s	904 B	73.2 KB	75.7 KB	164 K	3372 M	3282 M	24 B
EM-256f	1024 B	143.8 KB	148.3 KB	692 K	712 M	694 M	32 B
EM-256s	1024 B	143.8 KB	148.3 KB	672 K	5400 M	5252 M	32 B

cycle count for the 128f variant by approximately 40 million cycles.

The masked implementation uses 82.5 KB of memory for the signing algorithm, which is a 1.93x increase in memory usage compared to the unprotected implementation. This is as expected due to the Boolean masking of all secret values increasing the memory consumption linearly with the masking order. The performance of masked signing is 235 M cycles, which is an increase of 1.26x over the 187 M cycles for unprotected signing in the 128f variant. This is significantly lower than the asymptotic quadratic overhead of the secure multiplication from [RP10]. The lower performance penalty factor is due to the unmasked VOLEitH computation corresponding to the majority of the running time.

A comparison between the embedded implementation of FAEST and other post quantum signature schemes can be found in Table 3. The embedded implementation of FAEST does not have a memory consumption which is significant worse than other implementations of the signature scheme for both signing and verification. When directly comparing to the NIST standardized schemes (Dilithium, SPHINCS and Falcon), embedded FAEST is in the mix with memory consumption. Focusing on speed of the signature scheme, FAEST has by far the fastest key generation of all, and similar speed during signing and verification with other non-lattice based schemes. The lattice based schemes do have performance improvement in 2 orders of magnitudes compared to embedded FAEST.

5.1.3 Memory-Speed Trade-off on Embedded

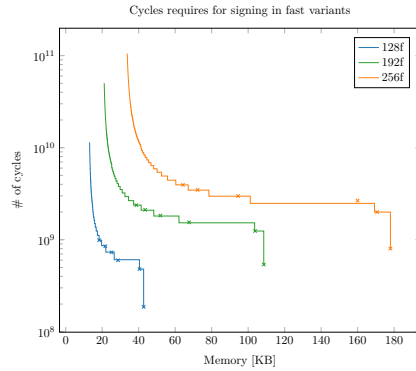
The embedded implementation uses the VBB, which enables it to decrease the memory consumption by partially recomputing the VOLE. This gives an interesting relation between the decreased memory consumption and the increased computation time.

The graph in Figure 4 shows the relationship for all available memory sizes. The line resembles the extrapolated data for all memory sizes, where the marks are experimentally observed data points. Due to the linearization of the access pattern of the VOLE, the performance resembles an exponential decay. Each jump in the cycle count is due to an increase in the needed number of recomputations of the VOLE, where each recomputation has an average cost of 127 M cycles for the 128f variant. Reducing the cache to the minimal size, a signature can be computed using only 13 KB of memory, but with a impractical cost of 11.4 billion cycles. The data shows that a significant additional decrease in memory is possible by paying the cost of a few recomputations, which gives an additional flexibility

Table 3: Memory usage and CPU cycles on the target platform (Cortex M4). All signature schemes are the fastest level 1 security variant presented, where the citation is the source of the benchmark. The masked variants are all first-order masked implementations.

Scheme		Stack Consumption			CPU Cycles		
		KeyGen	Sign	Verify	KeyGen	Sign	Verify
128f	[This work]	808 B	42.7 KB	41.4 KB	82.5 K	187 M	179 M
Dilithium	[KRSS19]	38.3 KB	49.4 KB	36.1 KB	1.87 M	7.28 M	2.06 M
SPHINCS- α	[KRSS19]	281 KB	281 KB	281 KB	30.2 M	382 M	35.7 M
Falcon-512	[Por25]	14.3 KB	42.0 KB	2.95 KB	71.9 M	22.0 M	255 K
Picnic3	[ABE ⁺ 21]	–	32.5 KB	–	–	302 M	–
MQOM	[KRSS19]	184 KB	399 KB	253 KB	7.79 M	149 M	137 M
PERK	[BBB ⁺ 24]	9.03 KB	25.1 KB	21.8 KB	0.82 M	240 M	78.1 M
UOV-Ip	[KNO ⁺ 24]	4.48 KB	5.23 KB	–	390 M	7.13 M	–
128f-masked	[This work]	732 B	82.5 KB	41.6 KB	78.6 K	235 M	179 M
Picnic3-masked	[ABE ⁺ 21]	–	32.5 KB	–	–	460 M	–
UOV-Ip-masked	[KNO ⁺ 24]	5.80 KB	10.2 KB	–	1.05 B	25.6 M	–

for the embedded implementation, as the memory requirement can be tweaked to fit the specific use case.



Scheme	Memory		Cycles	
	Sign	Verify	Sign	Verify
Fastest	43 KB	41 KB	187 M	179 M
Smallest	13 KB	13 KB	11.4 B	13.8 B

Figure 4: Memory vs. speed tradeoff for the embedded implementation. The graph shows the cycle counts (in millions or billions) for the signing algorithm as a function of the memory consumption excluding the signature. Marks represent experimentally observed data points, and lines represent the extrapolated data. The table shows the optimal points minimizing memory consumption and latency. The optimal setting with respect to both memory and cycle count is the fastest variant.

5.2 Side Channel Analysis

To verify that our masked implementation of FAEST is side-channel resistant in practice, we have performed a TVLA on the signing algorithm to confirm absence of exploitable leakage. The TVLA was performed on power traces captured using the ChipWhisperer Husky with the STM32F415 target board. The traces were collected at a target frequency of 22.1MHz and a sampling rate of 1 sample per clock cycle using 8 bits of precision. We follow the FvR approach by fixing the secret key in one group of traces and having a random message for all traces.

We start the collection just before the invocation of H_3 and stop at the end of the signing algorithm. This also includes the VOLEitH and VOLEHash computations, which by design are not masked since they are randomized per signature. These computations

are included to ensure that they do not suffer from additional side-channel leakage, beyond the t bits a probing adversary can learn. In Figure 7 a TVLA on the non-masked implementation of sign is presented. The non-masked implementation shows significant leakage with relatively few traces, validated the test setup.

The FAEST specific S-box in Figure 3 must sample from the distribution $\{2^8\} \setminus \{0\}$. In practice this should be done by rejection sampling, which results in a non-constant-time implementation. This is not a issue since the timing of the signature computation only reveals the number of times this random value was sampled to zero. This variable timing will create variability between the sample point and execution point, creating a problem during the side-channel evaluation. To ensure a constant-time execution the sampling is implemented by sampling from $\{2^{64}\}$ and mapping down to $\{2^8\} \setminus \{0\}$. This sampling is not-uniform, as the value 1 has a slight bias. This bias can result in a false indication of leakage if the bias is too significant. However, since the statistical distance from the uniform distribution is at most 2^{-56} , an expected total of 2^{56} S-box evaluations needs to be observed before the bias would have a noticeable effect. Thus the biased selection of r does not interfere with the side-channel analysis.

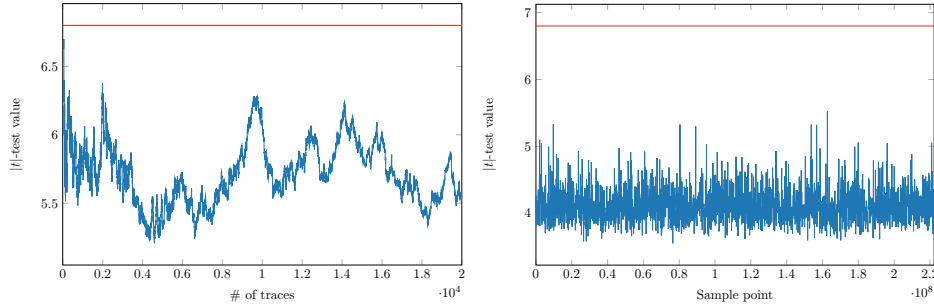


Figure 5: TVLA using Welch’s t -test of the FAEST signing algorithm based on 20,000 traces. The $|t|$ -values as a function of the number of traces collected (left) show the maximum $|t|$ -value is below the threshold of 6.8, with no tendency of increasing. The $|t|$ -value for all sample points (right) is far below the threshold, indicating the absence of leakage.

A total of 20,000 traces of the signing algorithm were collected, and the results are shown in Figure 5. The threshold for the TVLA is set to 6.8 as the traces contain $2.2 \cdot 10^8$ sample points, described by [DZD⁺17, table 1(b)]. On the left plot, the maximum $|t|$ -value is shown as a function of the number of traces collected. The $|t|$ -value fluctuates around values far below the threshold and do not show any clear tendency to increase, which indicates that the implementation is without exploitable leakage. The right plot shows the final $|t|$ -value for all sample points in the signing algorithm. The $|t|$ -value seems to be relatively high across all sample point of the trace, however the plot is structured such that each data point is the maximum $|t|$ -value for a subsection of 100,000 sample points. This removes all the low $|t|$ -values, which artificially increase the average of the plot, making it more readable. The largest $|t|$ -value is 5.69 which is far below the threshold of 6.8 and thus strongly indicates the absence of exploitable leakage. The spikes of the $|t|$ -value for the full signature are placed across the whole computation, indicating noise instead of a point of interest with increased leakage.

It is important to note the magnitude of the experiment. As each trace contains roughly 217 million samples points, both the storage requirements and the time commitment to collect these traces are significant. At the maximum sustainable sample rate of the Husky, each trace takes around 15 seconds to collect including the overhead of transferring the result. For the full TVLA test including 20,000 traces, the collection of traces takes almost

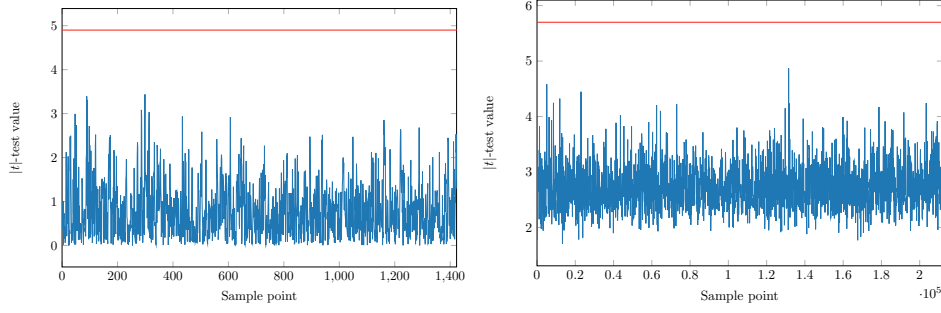


Figure 6: TVLA of the FAEST specific S-box gadget (left) and the Expand Witness algorithm (right), based on 500,000 and 100,000 traces respectively. The $|t|$ -values of both implementation stay below their threshold of 4.9 and 5.7 respectively, indicating the absence of exploitable leakage.

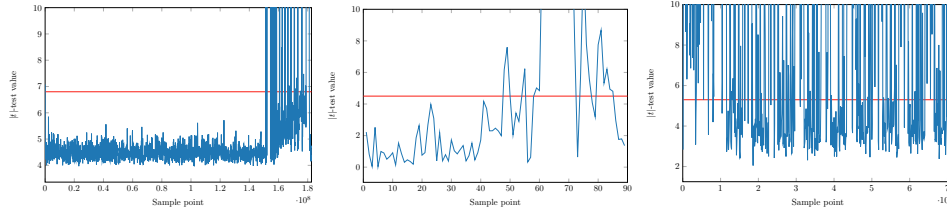


Figure 7: TVLA for setup validation of full signature computation(left), FAEST specific S-box gadget (middle), and Expand Witness computation (right). The TVLA is based on 1,000, 20,000, and 5,000 traces respectively, which threshold set according to [DZD⁺17, table 1(b)]. All non-masked implementation shows significant leakage across the computations.

7 days of continuously sampling. Moreover, storing the traces requires in total 8.7 TB of storage. Finally, performing the t -test analysis requires an additional day of continuous computation.

Individual parts. Due to the limits of our equipment and to further ensure the absence of leakage in the masked implementation, we have tested the S-box gadget and the Expand Witness function using a larger number of traces. The masked AESProve has not been tested independently, since it requires the VOLE and chall_2 , hence almost computing the full signature. The test setup is again using the ChipWhisperer Husky with the STM32F415 target board in the FvR approach. The traces are collected using 22.1 MHz and a sampling rate of 2 samples per clock cycle for the S-box gadget and 1 sample per clock cycle for the Expand Witness. The number of traces collected is increased to 500,000 for the S-box gadget and 100,000 for the Expand Witness. The results are shown in Figure 6. A setup validating TVLA is performed on the unmasked implementation of the S-box and Expand Witness functions shown in Figure 7.

The thresholds for the TVLA are set to 4.9 and 5.7 for respectively the S-box gadget and Expand Witness, as suggested by [DZD⁺17, table 1(b)] corresponding to traces with length 10^3 and 10^5 respectively. As both TVLA test stay below the threshold, the experiment indicates the absence of exploitable side-channel leakage. We present these tests to strengthen the results for the masked implementation, as the parts manipulating the secret key do not show signs of leakage, which strongly suggests that the full signature will behave likewise if more traces were to be collected.

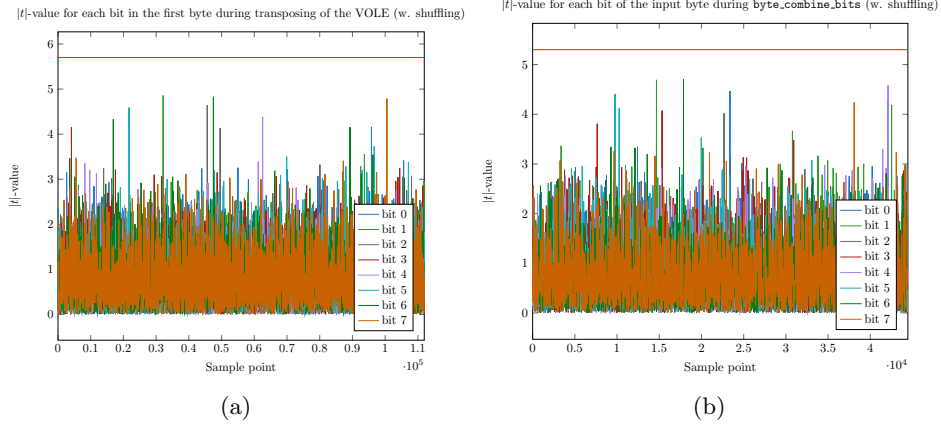


Figure 8: TVLA of transposing of the VOLE and `byte_combine_bits` with countermeasures applied. The $|t|$ -values of the first 8-bits on 100.000 traces are shown for transposing of the VOLE in Plot (a) and `byte_combine_bits` in plot (b). The $|t|$ -value is below of the threshold of 5.7 and 5.3 respectively [DZD⁺17, table 1(b)]. This indicates the absence of leakage.

5.3 Countermeasures to Horizontal Analysis

To experimentally verify the countermeasures the horizontal analysis from Section 4.5 two different verification processes have been applied. First, a Random vs Random (RvR) TVLA is performed to verify the absence of leakage in the shuffled implementations. In this test the traces are grouped based on the value of the selected bit in the random input. The experiments are performed on the ChipWhisperer Husky with the STM32F415 target board. The target was clocked at 22.1 MHz and the traces were collected with a sampling rate of 4 sample per clock cycle using 8-bit precision.

A machine learning attack identical to [JD25] is performed on the implementations with shuffling disabled. The model takes as input the power consumption trace of the function with the countermeasure applied, and tries to predict the value of the secret share. The inability to predict the secret shares indicates that the countermeasure is effective against such an attack.

Shuffling countermeasures. We first present the results of the implementation using both shuffling and countermeasures in Figure 8. A total of 16 groups are defined each based on the value of one of the bit in the target value.

The TVLA in Figure 8a shows the transposing of the VOLE. The results strongly indicate absence of leakage, as the maximum $|t|$ -value is below the threshold of 5.7 for all 8 bits. This is partly due to the shuffling changing the order of operations for each trace. However, as the TVLA also include the computation of the shuffling, it is clear that the leakage is not present on either entry or exit of the transposing. Since all 16 bytes of each OLE are transposed using the same assembly function, their leakage behavior will be identical to the first byte.

Furthermore, we present the results for the `byte_combine_bits` countermeasure in Figure 8b. The traces include the shuffling, all the dummy calls and the call with a secret share as input. The grouping of traces and TVLA is performed as before on each individual bit of the input. As the $|t|$ -value is below the threshold of 5.3, the countermeasure is effective and indicates the absence of leakage.

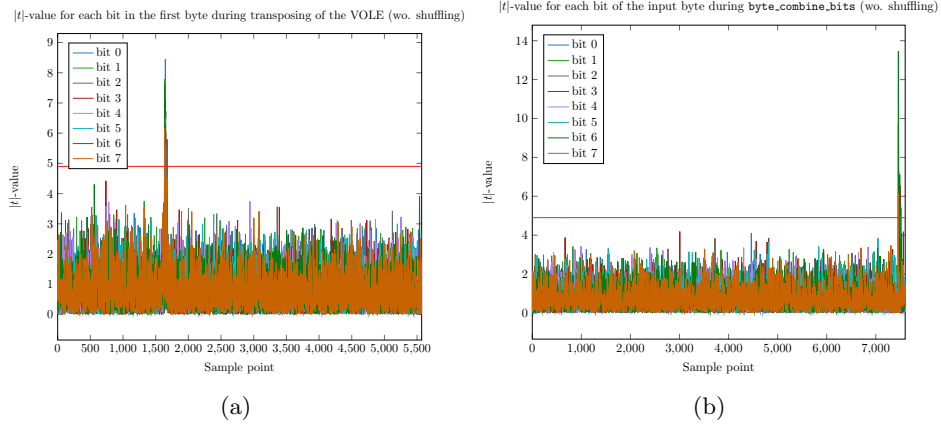


Figure 9: TVLA of transposing of the VOLE and `byte_combine_bits` with shuffling disabled. The $|t|$ -values of the first 8-bits on 5.000 traces are shown for transposing of the VOLE in Plot (a) and `byte_combine_bits` in plot (b). Both $|t|$ -value is above the threshold of 4.9 [DZD⁺17, table 1(b)]. The appears during the storage of the result, where all bits leak simultaneously

Computation countermeasures. Related work shows that the shuffling becomes an additional attack surface for the adversary [BNGD22], because the adversary can attempt to either align the traces or extract the permutation to circumvent the shuffling. The combinatorial complexity of the shuffling prevents efficient alignment of the traces, as discussed in Section 4.5. We perform a similar TVLA on both transposing of the VOLE and `byte_combine_bits` with the shuffling disabled to show that extracting the permutation does not pose a risk. This makes all the operations aligned across traces, and thus leakage depending on a bit will be visible in the t -test. The results are presented in Figure 9.

Both TVLA for transposing of the VOLE in Figure 9a and `byte_combine_bits` in Figure 9b show a significant leakage for each bit. Due to the nature of the test i.e. the grouping of traces based on the value of a bit, the t -test is likely to have a false positive. This is due to group where the bit is equal to 1 will have a higher average Hamming weight than the group where the bit is equal to 0. Looking at the case for the transposing of the VOLE, the large spike in the t -test is during the storage operation of the transposed OLE. Here all 8 bits leak at the same sample point, hence the leakage is the Hamming of the transposed value and not the individual bits, thus a false positive. The implementation combines 4 bytes into a machine word before storage to get the maximum possible Hamming weights, which increases the difficulty of distinguishing individual Hamming weights. In conclusion, the countermeasures applied to the transposing of the VOLE are effective and prevent horizontal analysis.

Looking at the `byte_combine_bits` in Figure 9b, the leakage is once more at the storage of the result. The bit leakage is again overlapping on the same sample points, thus the Hamming weight of the result leaks, which in this case is a 128-bit value. As the output of the function is uniquely determined by the input, there exists a direct correlation between the input and the output. Thus, the two groups used in the t -test will have disjoint output sets, which with enough traces can easily be seen in the t -test. However, the leakage is the Hamming weight of the 128-bit output value, so the adversary must be able to distinguish the Hamming weight of the output value to reduce the search space of the share.

Table 4: Success probability of the machine learning model on the reference implementation (ref. in the table) and the embedded implementation with countermeasures (Countermeasure in the table). The success probability is the probability the model predicts the correct value of the bit in the secret value. The prediction is performed on the validation set.

Probability				
Bit	byte_combine_bits		VOLE Transpose	
	Ref.	Countermeasure	Ref.	Countermeasure
0	0.52	0.51	0.91	0.54
1	1.00	0.50	0.80	0.52
2	1.00	0.50	0.77	0.54
3	0.99	0.50	0.74	0.53
4	1.00	0.50	0.74	0.54
5	0.99	0.50	0.74	0.53
6	0.99	0.50	0.70	0.53
7	0.99	0.50	0.65	0.53

5.3.1 Validation using Machine Learning Models

The countermeasures applied to the functions affected by horizontal analysis are effective at preventing leakage of the secret key, by avoiding bit-dependent leakage and performing shuffling of operations. As a probing adversary now has to probe both the permutation and the secret shares, the attack becomes a higher-order probing attack, which is outside the threat model of our masked implementation.

However, a horizontal attack based on machine learning models, such as [BNGD22], can attack both the shuffling and the function call without any sophisticated equipment or extraordinary effort. These types of attacks cannot be neglected due to their effectiveness, so we perform the same attack on the implementations of `byte_combine_bits` and the transposing of the VOLE with shuffling disabled using the same machine learning model as the original attack paper [JD25]. The model is trained on the power consumption traces of the function called on both shares of the secret value, and then tries to predict the value of the secret share. The power consumption traces were collected using the ChipWhisperer Husky with the STM32F415 target board. The target was clocked at 22.1 MHz and the traces were collected with a sampling rate of 4 sample per clock cycle using 8-bit of precision. The model was trained on 800,000 traces with 30% used for validation.

The model for the attack on `byte_combine_bits` has minor difference with respect to the model from the original paper [JD25]. We train 8 different models to try to predict each bit of the secret byte, instead of a single model used 8 times to predict each individual bits. This is done because there is not clear point of interest for each individual bit in the power consumption trace (as indicated in the *t*-test), hence the model gets access to all information available. The results can be seen in Table 4. To ensure that modifying the ML model is not the reason to an unsuccessful attack, we use the same model to attack the reference implementation. As the results show, the modified model can extract each bit during the reference implementation with almost perfect accuracy, while the attack on the embedded implementation is no better than random guessing.

A similar machine learning model is trained on the transposing of the VOLE implementation with shuffling disabled. A total of 8 models are trained, which try to predict the value of each bit of the secret value. Again to ensure the model is not the reason for an unsuccessful attack, the model is also trained on the reference implementation. The results in Table 4 show that the model has a very high accuracy on the reference implementation, with a slight decrease in accuracy for each additional bits. This decrease in accuracy happens because no bits are loaded and only 1 bit is stored when working with bit index 0, and a total of 7 bits are loaded and 8 bits are stored when working with bit index

8, hence the power consumption depends on more and more bits as the index increases. The accuracy of the model on the embedded implementation with countermeasures is just slightly better than random guessing, showing the countermeasure is effective against the attack.

5.4 Summary of the Side-Channel Analysis

The side-channel analysis of the embedded implementation shows that the implementation is secure against first-order probing attacks. This is shown by a TVLA on the full signature computations alongside TVLA on individual parts with an order of magnitude more traces than the full signature, showing the absence of exploitable first-order leakage. Furthermore the countermeasure against horizontal attacks on the transposing of the VOLE and `byte_combine_bits` are verified, both to remove the leakage of the individual bits and to make the ML model attack ineffective.

6 Applicability to FAEST 2.0

FAEST was selected as a Round 2 NIST candidate for additional signature schemes, which had a deadline for submissions in February 2025. In the weeks following the submission deadline, the FAEST team released a revised specification of FAEST, called FAEST v2, which includes multiple new optimisations. In this section we will discuss how the new specification of FAEST affects the work within the paper, and how the results from the paper are applicable to the new specification.

The masked embedded implementation of FAEST presented in this paper is based on version 1.1 of the specification. FAEST 1.1 has received public scrutiny for around 2 years, and without any relevant decrease in security, so our results build on a more tested and stable version of the signature algorithm. The whole process of going from the reference implementation to a secure masked embedded implementation took approximately 14 months, and this can now be greatly sped up for future versions of FAEST.

FAEST v2 changes many parts of the signature computation, namely, the vector commitment, the assumptions on the S-box and the ZK proof. First, the VC was replaced with a batched vector commitment (BaVC), where the τ trees of size N are combined into one single tree of size τN . The leaves of the original trees are interleaved, such that the first τ leaves of the BaVC tree correspond to the first leaf from each of the τ VC trees. The principle of the tree-walk algorithm presented in Section 3.1.1 is still applicable, but instead of computing each column of the VOLE matrix one-by-one, all columns will be computed in interleaved fashion, since the seeds of the tree are interleaved. This will have no impact on computing the vector commitments and their openings, and only a minor impact on memory, due to the tree-walk having to store a path of length $\log \tau + \log N$ instead of $\log N$, for the bigger tree.

When accessing the VOLE during the VOLEHash algorithm, the BaVC may incur a small performance penalty. If the VBB cache size is not enough to store the entire VOLE matrix, and we apply VOLEHash column-by-column as described in Section 3.1.1, there will be a performance penalty as the tree-walk algorithm has to skip over the leaves that are not needed to compute the relevant column. The performance penalty will depend on the VBB cache size, which dictates how many columns of the VOLE matrix can be processed at once, but will be at worst an $O(\log N)$ factor overhead. An alternative approach, which avoids re-computing of the leaves, is to compute all λ column hashes at once, by keeping track of λ hash values during the tree traversal. We take a single pass over the tree and fully expand each leaf on the way, incrementally adding each block of the expanded leaf to the relevant hash values. This requires $O(\lambda^2)$ storage for the λ hash values (which are each of size $\lambda + 16$ bits), but avoids any unnecessary re-computation.

When accessing VOLE in the ZK proof phase, the new BaVC will not have any performance overheads, as accessing each VOLE requires an entire row of the VOLE matrix to be computed, so there is no cost from the fact that we now have to compute multiple columns at a time. We therefore obtain the same memory reduction capabilities of the VBB in this phase.

Additionally, the assumption on the S-boxes has changed, since FAEST v2 no longer samples keys conditioned on all S-box inputs being non-zero. In the witness expansion, this makes our FAEST-specific S-box masking gadget obsolete. In the literature, there are algorithms for the S-box that can handle zero input [RP10]. Such an implementation is based on computing the inverse of x by computing the power x^{254} which requires quadratic number of multiplications with respect to the masking order. This will incur some performance overhead, however, as the expand witness computation is less than 1% of the total signing time, the penalty should be minimal.

Finally, the ZK proof has been changed. The FAEST v1.1, the AES computation is proved by showing consistency between the state in round i and round $i + 1$. In FAEST v2, the main proof is done by showing consistency between round $2i$ and round $2i + 2$ using a set of degree-3 constraints, which reduces the witness size and hence signature size. Using the VOLE, the prover commits to the state of round $2i + 1$ for every i , together with some smaller hints for rounds $2i$, that allow the verifier to check consistency. While this computation is more complex, and computing the constraints may require slightly more memory, we can still preserve the linear access pattern to the VOLEs. This is because the new ZK proof still manipulates the authenticated state round-by-round and uses a linear combination of all constraints at the end. Even though FAEST v2 has more constraints than v1, we can still avoid having to store them all in memory, obtaining roughly a factor R saving, where R is the number of rounds in AES.

In conclusion, the masked implementation of FAEST v1.1 can potentially be adapted to FAEST v2 without any surprises with respect to performance or memory requirements.

7 Conclusion and Future Perspectives

In this paper we have presented a memory optimized algorithm for VOLEitH, which is applicable for all VOLEitH based signature schemes. We achieve a memory decrease by a factor $O(\tau n / \log n)$ from the reference algorithm, with only an increase of $O(\log n)$ computation complexity. In practice this reduction ranges from 10 to 100 times less memory usage across all variants, and furthermore increases performance on a modern laptop CPU due to better placement in the memory hierarchy.

The reduced memory consumption allows for the implementation of the VOLEitH based signature scheme FAEST on resource-constrained devices, such as the Cortex-M4 microprocessor. Based on this implementation, we present a simple probing attack on the FAEST signature scheme. The probing attack leaks the secret key and motivates a masked version of the embedded implementation, which is protected against side-channel leakage. Beyond validating the security of the masked implementation using TVLA, we propose and implement additional countermeasures against known bit-by-bit and bit mask side-channel attacks in the reference FAEST code, by applying shuffling on top of masking. Since most computation of FAEST is per signature random, the side-channel protections have a performance overhead of 1.26x and a memory overhead of 1.93x.

This is the first embedded implementation of a VOLEitH-based signature scheme, showcasing techniques to reduce the memory consumption, and in return improving the performance on modern CPUs and allowing for computation on restricted hardware. This work also presents the first masked implementation of a VOLEitH-based signature scheme and the FAEST signature scheme. This work can be used as a starting point for future research on the VOLEitH construction both in terms of performance, masking and

portability to different architectures. As the VOLEitH construction itself is out of the scope under our threat model, it could be of interest to investigate how the construction can be made side-channel secure in a compatible way with the reference.

References

- [ABE⁺21] Diego F. Aranha, Sebastian Berndt, Thomas Eisenbarth, Okan Seker, Akira Takahashi, Luca Wilke, and Greg Zaverucha. Side-channel protections for picnic signatures. *IACR TCHES*, 2021(4):239–282, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/9066>, doi:10.46586/tches.v2021.i4.239-282.
- [ACLZ20] Dorian Amiet, Andreas Curiger, Lukas Leuenberger, and Paul Zbinden. Defeating NewHope with a single trace. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 189–205. Springer, Cham, 2020. doi:10.1007/978-3-030-44223-1_11.
- [AP21] Alexandre Adomnicaï and Thomas Peyrin. Fixslicing AES-like ciphers. *IACR TCHES*, 2021(1):402–425, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/8739>, doi:10.46586/tches.v2021.i1.402-425.
- [BBB⁺24] Slim Bettaieb, Loïc Bidoux, Alessandro Budroni, Marco Palumbi, and Lucas Pandolfo Perin. Enabling perk and other mpc-in-the-head signatures on resource-constrained devices. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):84–109, Sep. 2024. URL: <https://tches.iacr.org/index.php/TCHES/article/view/11784>, doi:10.46586/tches.v2024.i4.84-109.
- [BBD⁺23] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023. doi:10.1007/978-3-031-38554-4_19.
- [BBdSG⁺23] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. *FAEST: Algorithm Specifications*, version 1.1 edition, July 2023. URL: <https://faest.info/faest-spec-v1.1.pdf>.
- [BBGK24] Slim Bettaieb, Loïc Bidoux, Philippe Gaborit, and Mukul Kulkarni. Modelings for generic PoK and applications: Shorter SD and PKP based signatures. *Cryptology ePrint Archive*, Report 2024/1668, 2024. URL: <https://eprint.iacr.org/2024/1668>.
- [BBM⁺24] Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 463–493. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0875-1_15.

- [BFR23] Ryad Benadjila, Thibault Feneuil, and Matthieu Rivain. Mq on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem. Cryptology ePrint Archive, Paper 2023/1719, 2023. <https://eprint.iacr.org/2023/1719>. URL: <https://eprint.iacr.org/2023/1719>.
- [BNGD22] Linus Backlund, Kalle Ngo, Joel Gärtner, and Elena Dubrova. Secret key recovery attacks on masked and shuffled implementations of CRYSTALS-Kyber and Saber. Cryptology ePrint Archive, Report 2022/1692, 2022. URL: <https://eprint.iacr.org/2022/1692>.
- [Bui24] Dung Bui. Shorter VOLEitH signature from multivariate quadratic. Cryptology ePrint Archive, Report 2024/465, 2024. URL: <https://eprint.iacr.org/2024/465>.
- [CDG⁺20] Melissa Chase, David Derler, Steven Goldfeder, Daniel Kales, Jonathan Katz, Vladimir Kolesnikov, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Xiao Wang, and Greg Zaverucha. The picnic signature scheme, version 3.0. 2020. <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [CFG⁺10] Christophe Clavier, Benoit Feix, Georges Gagnerot, Mylène Roussellet, and Vincent Verneuil. Horizontal correlation analysis on exponentiation. In Miguel Soriano, Sihan Qing, and Javier López, editors, *ICICS 10*, volume 6476 of *LNCS*, pages 46–61. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17650-0_5.
- [CGP⁺12] Claude Carlet, Louis Goubin, Emmanuel Prouff, Michaël Quisquater, and Matthieu Rivain. Higher-order masking schemes for S-boxes. In Anne Canteaut, editor, *FSE 2012*, volume 7549 of *LNCS*, pages 366–384. Springer, Berlin, Heidelberg, March 2012. doi:10.1007/978-3-642-34047-5_21.
- [CGTZ23] Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of dilithium. Cryptology ePrint Archive, Report 2023/896, 2023. URL: <https://eprint.iacr.org/2023/896>.
- [DZD⁺17] A. Adam Ding, Liwei Zhang, François Durvaux, François-Xavier Standaert, and Yunsu Fei. Towards sound and optimal leakage detection procedure. In Thomas Eisenbarth and Yannick Tegliah, editors, *Smart Card Research and Advanced Applications - 16th International Conference, CARDIS 2017, Lugano, Switzerland, November 13-15, 2017, Revised Selected Papers*, volume 10728 of *Lecture Notes in Computer Science*, pages 105–122. Springer, 2017. doi:10.1007/978-3-319-75208-2_7.
- [FRWJ25] Thibault Feneuil, Matthieu Rivain, and Auguste Warmé-Janville. Masking-friendly post-quantum signatures in the threshold-computation-in-the-head framework. Cryptology ePrint Archive, Paper 2025/520, 2025. URL: <https://eprint.iacr.org/2025/520>.
- [FY38] RA Fisher and F Yates. Statistical tables for biological, agricultural and medical research. 1938.
- [GD23] John Gaspoz and Siemen Dhooghe. Threshold implementations in software: Micro-architectural leakages in algorithms. *IACR TCHES*, 2023(2):155–179, 2023. doi:10.46586/tches.v2023.i2.155-179.

- [GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986. doi: [10.1145/6490.6503](https://doi.org/10.1145/6490.6503).
- [GJJR11] Gilbert Goodwill, Benjamin Jun, Joshua Jaffe, and Pankaj Rohatgi. A testing methodology for side channel resistance validation. 2011. URL: <https://api.semanticscholar.org/CorpusID:16852899>.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003. doi: [10.1007/978-3-540-45146-4_27](https://doi.org/10.1007/978-3-540-45146-4_27).
- [JD25] Sönke Jendral and Elena Dubrova. Side-channel and fault injection attacks on VOLEitH signature schemes: A case study of masked FAEST. Cryptology ePrint Archive, Paper 2025/378, 2025. URL: <https://eprint.iacr.org/2025/378>.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999. doi: [10.1007/3-540-48405-1_25](https://doi.org/10.1007/3-540-48405-1_25).
- [KNO⁺24] Suparna Kundu, Quinten Norga, Uttam Kumar Ojha, Anindya Ganguly, Angshuman Karmakar, and Ingrid Verbauwhede. mUOV: Masking the unbalanced oil and vinegar digital signature scheme at first- and higher-order. Cryptology ePrint Archive, Report 2024/1875, 2024. URL: <https://eprint.iacr.org/2024/1875>.
- [KRSS19] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. Cryptology ePrint Archive, Report 2019/844, 2019. URL: <https://eprint.iacr.org/2019/844>.
- [LD00] Julio Cesar López-Hernández and Ricardo Dahab. High-speed software multiplication in $\mathbb{F}_{2^m}$. In Bimal K. Roy and Eiji Okamoto, editors, *INDOCRYPT 2000*, volume 1977 of *LNCS*, pages 203–212. Springer, Berlin, Heidelberg, December 2000. doi: [10.1007/3-540-44495-5_18](https://doi.org/10.1007/3-540-44495-5_18).
- [LDK⁺22] Vadim Lyubashevsky, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehle, and Shi Bai. Crystals-dilithium. Technical report, National Institute of Standards and Technology, 2022. <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [OTX24] Ying Ouyang, Deng Tang, and Yanhong Xu. Code-based zero-knowledge from VOLE-in-the-head and their applications: Simpler, faster, and smaller. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 436–470. Springer, Singapore, December 2024. doi: [10.1007/978-981-96-0935-2_14](https://doi.org/10.1007/978-981-96-0935-2_14).
- [Por25] Thomas Pornin. Falcon on ARM cortex-m4: an update. Cryptology ePrint Archive, Paper 2025/123, 2025. URL: <https://eprint.iacr.org/2025/123>.

- [QS01] Jean-Jacques Quisquater and David Samyde. Electromagnetic analysis (EMA): measures and counter-measures for smart cards. In Isabelle Attali and Thomas P. Jensen, editors, *Smart Card Programming and Security, International Conference on Research in Smart Cards, E-smart 2001, Cannes, France, September 19-21, 2001, Proceedings*, volume 2140 of *Lecture Notes in Computer Science*, pages 200–210. Springer, 2001. doi:10.1007/3-540-45418-7_17.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022. doi:10.1007/978-3-031-15802-5_23.
- [RP10] Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In Stefan Mangard and François-Xavier Standaert, editors, *CHES 2010*, volume 6225 of *LNCS*, pages 413–427. Springer, Berlin, Heidelberg, August 2010. doi:10.1007/978-3-642-15031-9_28.
- [SS25] David Santos and Michael Scott. Side-channel safe conditional moves and swaps. Cryptology ePrint Archive, Paper 2025/935, 2025. URL: <https://eprint.iacr.org/2025/935>.
- [Tea21] Team Keccak. XKCP – extended Keccak code package. <https://github.com/XKCP/XKCP>, 2021.
- [YCZ24] Yu Yu, Hongrui Cui, and Kaiyi Zhang. Sphincs-alpha. Technical report, National Institute of Standards and Technology, 2024. <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021. doi:10.1145/3460120.3484556.